

Final Project Report

Code Smells in the Wild

JGraphT Code-Quality Analysis,
Refactoring, and Functional Validation

Repository: github.com/jgrapht/jgrapht

Selected component: BoykovKolmogorovMFIml.adopt()

Course deliverable: Phase 5 Documentation

Date: June 25, 2026

JGraphT Code-Quality Analysis, Refactoring, and Functional Validation

Executive Summary

This project examined the open-source JGraphT Java graph library through repository setup, architecture study, manual and tool-based code-quality analysis, behavior-preserving refactoring, and functional validation.

The selected production target was the private `adopt()` method in `BoykovKoltmogorovMFImpL`. SonarQube reported a cognitive complexity of 112 and classified the method as a Brain Method. PMD and manual inspection additionally identified nested and duplicated source-tree and sink-tree conditional logic.

The committed refactor decomposes orphan selection, source-tree processing, sink-tree processing, parent search, free-vertex handling, and common adoption operations into focused private helpers. It changes no public API.

Functional validation showed:

Validation	Result
Original focused tests	31 passed
Refactored focused tests	31 passed
Refactored maximum-flow regression tests	102 passed
Failures, errors, or skipped tests	0

The available evidence demonstrates that the refactor improves internal structure while preserving the tested maximum-flow behavior.

1. Repository Overview

1.1 Project Information

Item	Details
Project	JGraphT
Project type	Open-source Java graph-theory library
Original repository	< https://github.com/jgrapht/jgrapht >
Personal fork	< https://github.com/Avraaaa/jgrapht >
Build system	Multi-module Maven
Required Java version	JDK 21 or later
Local environment used	Java 25.0.2 and Maven 3.9.16
Refactoring branch	<code>refactor-boykov-koltmogorov-adopt</code>

JGraphT provides graph data structures and algorithms as a reusable library. It is not a single application with one business-process entry point. Client programs create graphs, apply algorithms, traverse vertices, or import and export graph data through public APIs.

1.2 Module Architecture

The parent Maven project coordinates nine modules:

Module	Main responsibility
<code>jgrapht-core</code>	Public graph API, graph implementations, algorithms, traversal, generators, and utilities
<code>jgrapht-io</code>	Import and export formats including DOT, CSV, GraphML, JSON, and others
<code>jgrapht-osm</code>	OpenStreetMap road-graph integration

Module	Main responsibility
jgrapht-opt	Memory- and performance-oriented graph representations
jgrapht-ext	External adapters and visualization integration
jgrapht-guava	Adapters for Google Guava graph structures
jgrapht-unimi-dsi	WebGraph and succinct-graph integrations
jgrapht-demo	Runnable examples
jgrapht-dist	Distribution archive assembly

The architecture can be summarized as:

```
Parent Maven reactor
|
+-- jgrapht-core: central API and algorithms
    |
    +-- I/O and data modules: jgrapht-io, jgrapht-osm
    +-- optimized representations: jgrapht-opt, jgrapht-unimi-dsi
    +-- adapters: jgrapht-ext, jgrapht-guava
    +-- execution and packaging: jgrapht-demo, jgrapht-dist
```

1.3 Build and Execution

The repository was built through the demo module and its dependencies:

```
mvn -pl jgrapht-demo -am -DskipTests package
```

The runnable HelloJGraphT demo was used as the library execution path:

```
mvn -pl jgrapht-demo `
  "-Dexec.mainClass=org.jgrapht.demo.HelloJGraphT" `
  exec:java
```

The demo successfully exercised graph creation, depth-first traversal, and DOT export, ending with BUILD SUCCESS.

2. Identified Issues and Categorization

2.1 Analysis Strategy

Findings were collected with static-analysis tools and manual inspection. Tool findings were not accepted automatically. Each candidate was checked for:

- Whether it represented a real maintainability problem.
- Whether it affected production code or only tests.
- Whether a behavior-preserving refactor was practical.
- Whether the change could remain narrow enough for an open-source pull request.
- Whether existing tests could provide meaningful validation.

2.2 Main Findings

Category	Location	Evidence	Decision
Brain Method / Long Method	BoykovKolmogorovMFImpl.adopt()	SonarQube cognitive complexity 112 and Brain Method; PMD nested-condition findings; manual responsibility analysis	Selected and refactored
Duplicated conditional logic	Source-tree and sink-tree branches in adopt()	Similar parent search, adoption, free-vertex handling, activation, and child-orphan logic with different edge orientation	Selected and refactored
Long Method / conditional complexity	BoykovKolmogorovMFImpl.grow()	SonarQube cognitive complexity 68; source/sink duplication	Documented and deferred
Complex conditional logic	AhujaOrlinSharmaCyclicExchangeLocalAugmentation.getLocalAugmentationCycle()	SonarQube cognitive complexity 54 and nested-condition findings	Secondary candidate
Long Parameter List / Data Clumps	NetworkGeneratorConfig constructor	SonarQube reported 13 parameters	Safer alternative, not selected
Loop-counter mutation	NetworkGenerator methods	PMD and SonarQube findings	Valid but too small for the primary refactor
God Class / Too Many Methods	Several algorithm and generator classes	IntelliJ PMD findings	Broad class splitting rejected as too risky
Debug console output	Debug branches in BoykovKolmogorovMFImpl	SonarQube console-output findings	Outside selected scope
Cosmetic cleanup	Parentheses, imports, modifiers, and similar findings	PMD report	Not substantial enough for the main assignment

2.3 Selected Smell

The final selected smell was:

Brain Method / Long Method with duplicated source-tree and sink-tree conditional logic

The adopt() method mixed several algorithm steps:

- Selecting the next orphan.
- Finding a source-tree parent.
- Finding a sink-tree parent.
- Adopting a vertex.
- Making an unadoptable vertex free.
- Activating related vertices.
- Creating child orphans.
- Updating timestamps and distances.

This concentration made the method difficult to review and increased the risk of inconsistent changes between the source-tree and sink-tree branches.

3. Tools Used for Analysis and Validation

Tool	Purpose	Evidence produced
SonarQube Community Edition	Cognitive complexity, Brain Method, parameter count, and general code-smell detection	SonarQube logs and exported findings
PMD	Nested conditionals, loop behavior, naming/style, and maintainability rules	PMD text report and IntelliJ PMD screenshots
IntelliJ inspection / PMD integration	Visual inspection of reported class-level findings	Screenshots in Phase 2
Manual source inspection	Validate tool findings and analyze algorithm invariants	Manual-analysis and target-selection documents
Maven	Build modules and execute focused and regression tests	Build and execution logs
JUnit Platform	Verify maximum-flow behavior through the public API	31-test and 102-test results
Git	Isolate the refactor, inspect the diff, and compare revisions	Commit history and before/after revisions
LLM-assisted analysis	Organize large reports, compare candidates, and prepare documentation	Prompt logs and report drafts

The static-analysis output contained many findings, so manual inspection was essential for separating high-value maintainability problems from cosmetic warnings, false priorities, and changes unsuitable for a narrow refactoring contribution.

4. Refactoring Approach and Reasoning

4.1 Scope

The committed production change is:

```
jgrapht-core/src/main/java/org/jgrapht/alg/flow/BoykovKolmogorovMFImpl.java
```

Original revision:

```
3215ef7eebdbaaf11f95d92d614fc552e8d9006e
```

Refactored revision:

```
6183e894718be159d82a2182a72ca3bfb08b2a90
```

Commit:

```
Refactor Boykov-Kolmogorov orphan adoption
```

The diff changes one production file with 152 insertions and 105 deletions. No public interface or test file was changed.

4.2 Applied Refactorings

The main technique was Extract Method. The original adopt () method was reduced to coordination:

```
private void adopt()
{
    while (!orphans.isEmpty() || !childOrphans.isEmpty()) {
        VertexExtension currentVertex = nextOrphan();

        if (currentVertex.isSourceTreeVertex()) {
            adoptSourceOrphan(currentVertex);
        } else {
            assert currentVertex.isSinkTreeVertex();
            adoptSinkOrphan(currentVertex);
        }
    }
}
```

The following responsibilities were extracted:

Helper	Responsibility
<code>nextOrphan()</code>	Preserve child-orphan priority and select the next orphan
<code>adoptSourceOrphan()</code>	Coordinate source-tree orphan handling
<code>findSourceTreeParent()</code>	Search for the closest valid source-tree parent
<code>makeSourceTreeVertexFree()</code>	Free an unadoptable source-tree vertex and process neighbors
<code>adoptSinkOrphan()</code>	Coordinate sink-tree orphan handling
<code>findSinkTreeParent()</code>	Search for the closest valid sink-tree parent
<code>makeSinkTreeVertexFree()</code>	Free an unadoptable sink-tree vertex and process neighbors
<code>makeFree()</code>	Apply common free-vertex state changes
<code>adoptWithParent()</code>	Apply common adoption state changes
<code>AdoptionParent</code>	Carry the selected parent edge and distance together

4.3 Why Explicit Source and Sink Helpers Were Retained

The source-tree and sink-tree branches are structurally similar but not interchangeable:

- Source parent search checks inverse-edge capacity.
- Sink parent search checks forward-edge capacity.
- Source adoption stores an inverse edge.
- Sink adoption stores a forward edge.
- Source child detection compares against the forward edge.
- Sink child detection compares against the inverse edge.

Combining these branches into a highly generic abstraction could reduce repetition but would hide the directional rules that determine algorithm correctness. Explicit helpers improve navigation while leaving the important differences visible.

4.4 Scope Correction from the Phase 3 Proposal

The Phase 3 planning documents discussed refactoring both `grow()` and `adopt()`. The final commit changes only `adopt()`. It also uses an `AdoptionParent` helper to preserve the selected edge and distance together.

The final report follows the committed implementation rather than presenting unimplemented planning as completed work. The `grow()` complexity finding remains a documented future candidate.

5. Before vs After Comparison

5.1 Structural Comparison

Area	Before	After
Main <code>adopt()</code> responsibility	Selection, parent search, adoption, freeing, activation, and child-orphan handling	High-level orphan-processing loop and directional dispatch
Source-tree logic	Nested inline branch	Named source-specific helpers
Sink-tree logic	Nested inline branch	Named sink-specific helpers
Common state changes	Repeated in both branches	Extracted into <code>makeFree()</code> and <code>adoptWithParent()</code>
Parent result	Separate local edge and distance variables	<code>AdoptionParent</code> value holder
Public API	Existing API	Unchanged
Production files changed	Not applicable	One

5.2 Before Shape

The original method placed source and sink processing inside one large loop:

```
private void adopt()
{
    while (!orphans.isEmpty() || !childOrphans.isEmpty()) {
        VertexExtension currentVertex;
        // orphan selection

        if (currentVertex.isSourceTreeVertex()) {
            // source parent search
            // source adoption or free processing
        } else {
            // sink parent search
            // sink adoption or free processing
        }
    }
}
```

5.3 After Shape

The refactored method exposes the algorithm steps through helper names:

```
private void adopt()
{
    while (!orphans.isEmpty() || !childOrphans.isEmpty()) {
        VertexExtension currentVertex = nextOrphan();

        if (currentVertex.isSourceTreeVertex()) {
            adoptSourceOrphan(currentVertex);
        } else {
            assert currentVertex.isSinkTreeVertex();
            adoptSinkOrphan(currentVertex);
        }
    }
}
```

The result is longer at the file level because meaningful helpers were added, but the main procedure is easier to scan, each helper has a narrower purpose, and source/sink invariants can be reviewed independently.

6. Functional Validation Evidence

6.1 Validation Requirements

The refactor was required to preserve:

- Maximum-flow values.
- Public API signatures.
- Orphan and child-orphan processing order.
- Source-tree and sink-tree edge orientation.
- Residual-capacity checks.
- Parent selection by minimum distance.
- Timestamp and distance updates.
- Directed and undirected graph behavior.

6.2 Original and Refactored Focused Tests

The original revision was executed in a separate detached worktree. Both versions used:

```
mvn -pl jgrapht-core \
    "-Dtest=org.jgrapht.alg.flow.BoykovKolmogorovMFImplTest" \
    test
```

Result	Original	Refactored
Tests run	31	31
Failures	0	0
Errors	0	0
Skipped	0	0
Build	Success	Success

A representative test asserts maximum-flow values of 19 for a directed graph and 34 for the corresponding undirected graph. The test passed in both revisions.

6.3 Broader Regression Test

The refactored revision was also tested against selected maximum-flow implementations:

```
mvn -pl jgrapht-core \
    "-Dtest=org.jgrapht.alg.flow.*MFImplTest,org.jgrapht.alg.flow.MaximumFlowAlgorithmTest" \
    test
```

Results:

```
Tests run: 102
Failures: 0
Errors: 0
Skipped: 0
Build result: BUILD SUCCESS
```

The passing focused comparison and broader regression set provide executable evidence that the refactor preserves the tested external behavior.

7. Challenges Faced

7.1 Selecting a Suitable Open-Source Target

Many repositories either discourage refactoring-only pull requests or are so mature that obvious cleanup is cosmetic. The target needed to be meaningful, behavior-preserving, reviewable, and testable without changing public behavior. JGraphT provided mature tests and complex algorithm code, but required careful scope control.

7.2 Managing Large Static-Analysis Output

The SonarQube export contained thousands of unresolved findings, and PMD produced many repeated or low-value warnings. Raw finding counts were not useful by themselves. Findings had to be deduplicated, categorized, mapped to real source locations, and checked manually before selecting a target.

7.3 Understanding Algorithm-Specific Invariants

BoykovKolmogorovMFImpl is not ordinary application code. Source-tree and sink-tree logic uses opposite edge orientations. A mechanically symmetric refactor could compile while silently changing the algorithm. The main challenge was improving structure without changing edge identity, queue order, capacity checks, timestamps, or distances.

7.4 Balancing Abstraction and Readability

A generic abstraction could remove more duplicate structure, but it would make the directional differences harder to audit. The chosen design accepts separate source and sink helpers because correctness and reviewability are more important than minimizing line count.

7.5 Keeping Documentation Consistent with the Final Commit

The Phase 3 documents were produced while the design was still evolving. They proposed a broader `grow()` and `adopt()` refactor and discussed alternatives to `AdoptionParent`. The final commit was narrower. Phase 4 and this report therefore had to distinguish planned work from actual completed work.

7.6 Reproducing the Original Baseline

Functional comparison required running the original and refactored revisions independently. A detached Git worktree was used for the original revision so that the active refactoring branch did not need to be modified. Maven also needed to resolve an uncached plugin before the final validation runs could proceed.

8. Reflection on Improving Real-World Code Quality

This project demonstrated that improving real-world code quality is not equivalent to fixing every static-analysis warning. Mature projects contain deliberate complexity, compatibility constraints, algorithm-specific design, and established contribution expectations.

The most important lessons were:

- Tool findings are starting points, not final conclusions.
- A smaller refactor with strong tests is more defensible than a broad redesign.
- Public API stability is especially important in a library.
- In algorithm code, readability includes making invariants visible rather than maximizing reuse.
- Before/after tests should be planned before editing, not added only after the change.
- Documentation must describe the final diff accurately, including work that was intentionally

deferred.

- More helper methods can still improve maintainability when they separate responsibilities and

expose the algorithm's conceptual stages.

The refactor did not attempt to eliminate all complexity from the Boykov-Kolmogorov implementation. Instead, it moved complexity into smaller, named units while preserving the source/sink distinctions that a future maintainer must understand. This is a practical form of code-quality improvement: reduce the cognitive load of safe modification without pretending the underlying algorithm is simple.

9. Conclusion

The project completed repository setup, architecture analysis, tool-based and manual code-quality analysis, a focused production refactor, and before/after functional validation.

The selected adopt () Brain Method was decomposed into focused private helpers without changing the public API. The original and refactored revisions both passed all 31 focused tests, and the refactored revision passed 102 broader maximum-flow tests. No failures, errors, or skipped tests were reported.

The completed work therefore satisfies the Phase 5 documentation requirements and provides a traceable account of the repository, identified issues, analysis tools, refactoring decisions, before/after structure, functional evidence, challenges, and lessons learned.

Appendix A: Submission Evidence Index

Phase	Main evidence
Phase 1	Phase1/phase1_report.md, architecture diagrams, build and demo logs
Phase 2	Phase2/phase2_report.md, PMD and SonarQube reports, screenshots, manual analysis
Phase 3	Before-change baseline, planned comparison documents, committed Git refactor
Phase 4	Original/refactored logs, 102-test regression log, functional comparison report
Phase 5	Phase5/final_report.md and Phase5/phase5_final_report.pdf